

UNIVERSIDAD CENFOTEC
ESCUELA DE INGENIERÍA DEL SOFTWARE

PRINCIPIOS DE PROGRAMACIÓN 1

MISIÓN ÓRBITA

Primer Avance del Proyecto: Preparación de la Misión

Integrantes del grupo: Katherine Angulo Angulo
Docente : Verónica Mora Lezcano
Código del curso: SOFT-01
Período: C3-2026
Fecha de entrega: 14 de octubre de 2026

Repositorio de GitHub del proyecto:

https://github.com/kanguloan-cloud/mision-orbita

Ejercicio 1: Cálculo de recursos necesarios para una misión

La consigna establece que este ejercicio debe solicitar **nombre de la misión, cantidad de tripulantes y días estimados para llegar al destino**, y luego calcular combustible, oxígeno, energía y provisiones.

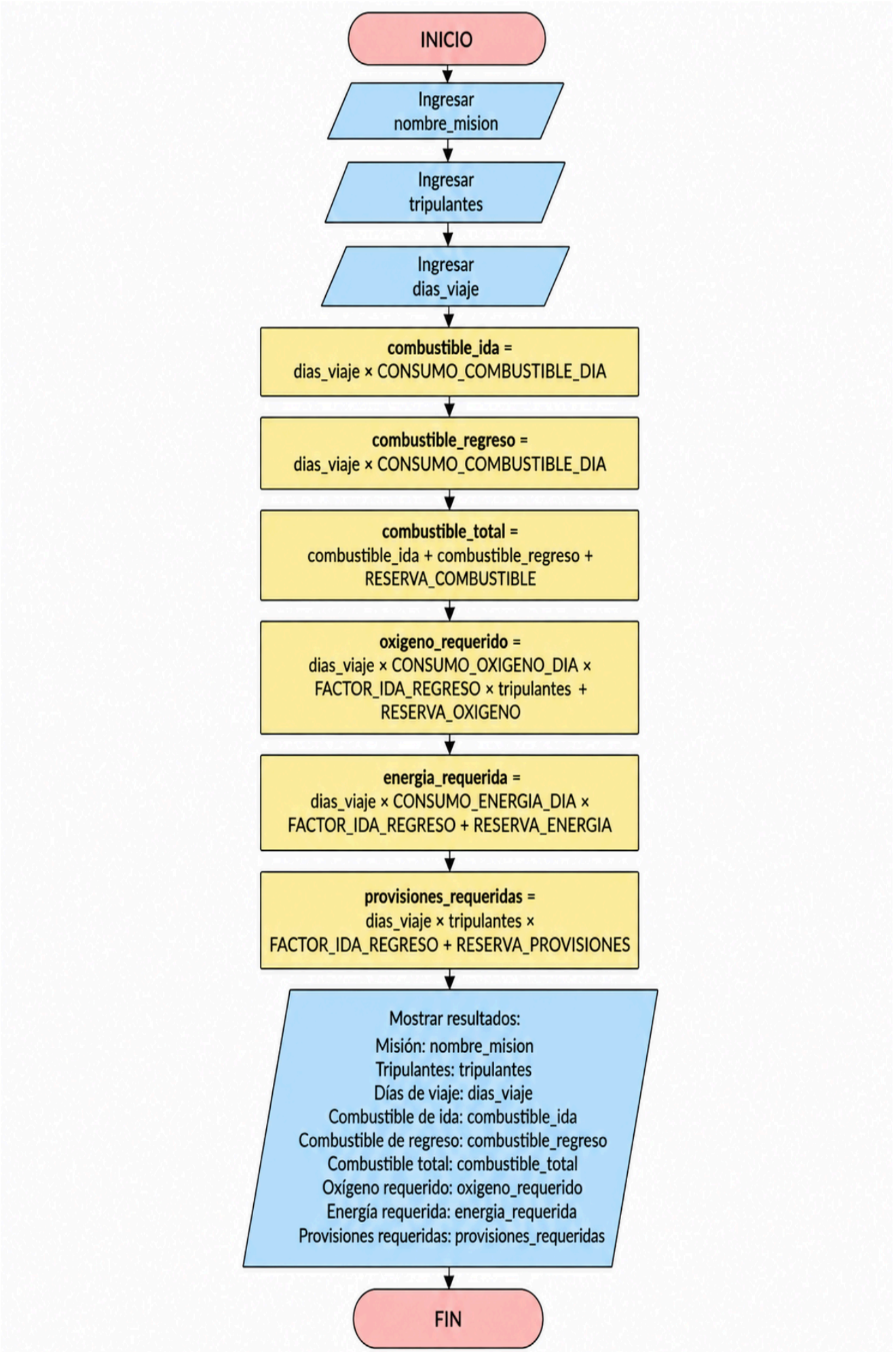
Análisis del problema

El programa permitirá calcular los recursos necesarios para realizar una misión espacial. Para ello, solicitará el nombre de la misión, la cantidad de tripulantes y la cantidad de días estimados para llegar al destino. A partir de estos datos se calcularán el combustible necesario para el viaje de ida y regreso, la reserva de combustible, el oxígeno, la energía y las provisiones requeridas para la misión.

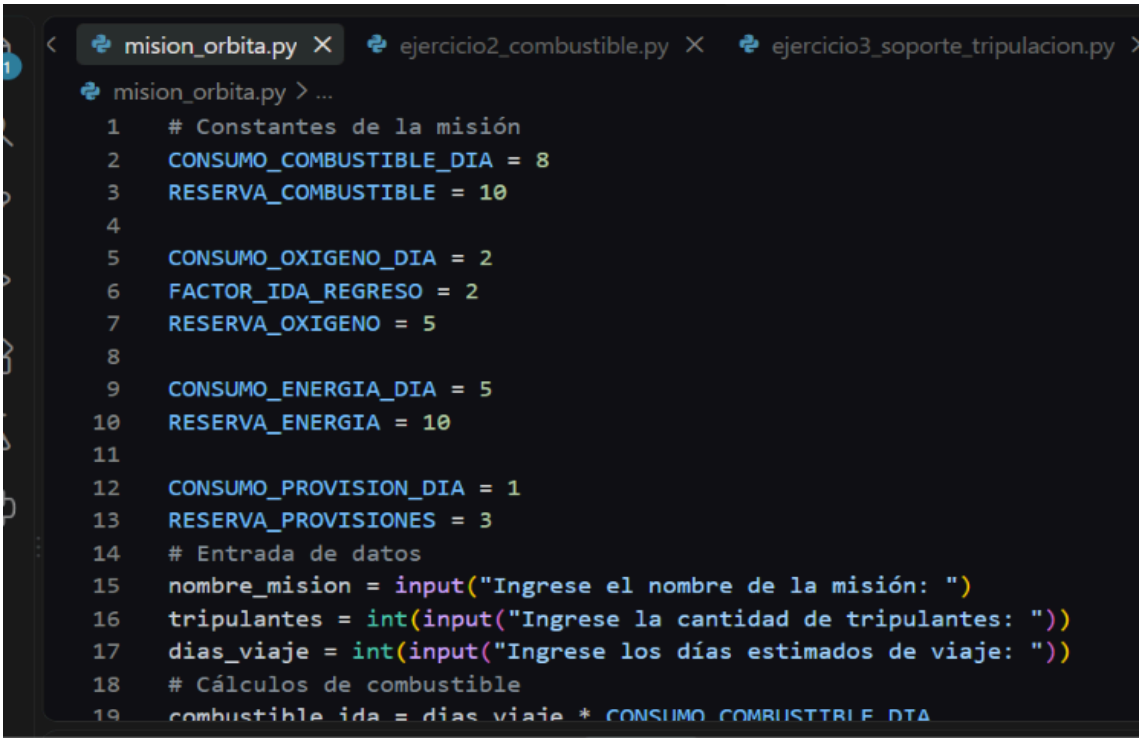
TABLA DE VARIABLES

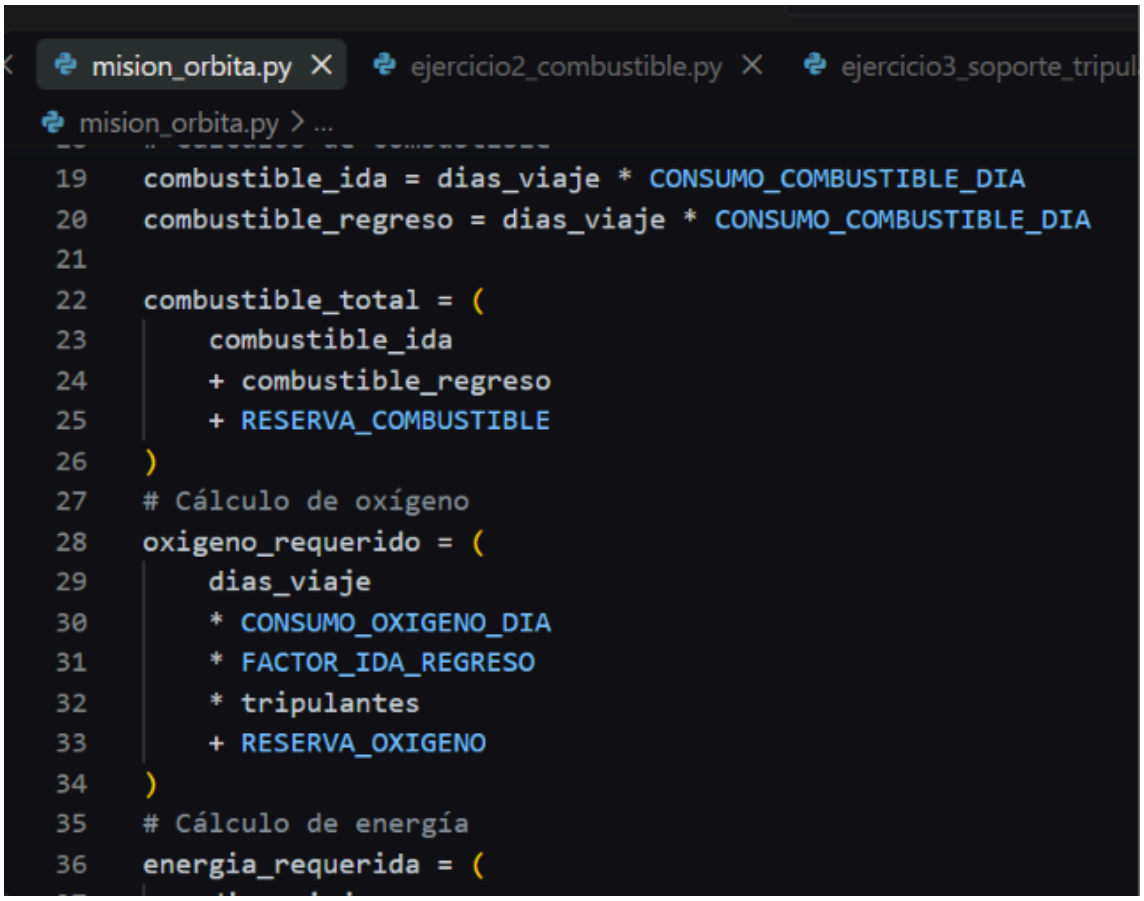
Nombre	Tipo	Clasificación	Descripción
nombre_mision	str	Entrada	Nombre de la misión.
tripulantes	int	Entrada	Cantidad de tripulantes.
dias_viaje	int	Entrada	Días estimados para llegar al destino.
CONSUMO_COMBUSTIBLE_DIA	int	Constante	8 unidades de combustible por día.
RESERVA_COMBUSTIBLE	int	Constante	10 unidades adicionales de combustible.
CONSUMO_OXIGENO_DIA	int	Constante	2 unidades de oxígeno por tripulante y día.
FACTOR_IDA_REGRESO	int	Constante	Factor 2 para ida y regreso.
RESERVA_OXIGENO	int	Constante	5 unidades adicionales de oxígeno.
CONSUMO_ENERGIA_DIA	int	Constante	5 unidades de energía por día.
RESERVA_ENERGIA	int	Constante	10 unidades adicionales de energía.
CONSUMO_PROVISION_DIA	int	Constante	1 unidad por tripulante y día.
RESERVA_PROVISIONES	int	Constante	3 unidades adicionales de provisiones.
combustible_ida	int	Intermedia	Combustible para el viaje de ida.
combustible_regreso	int	Intermedia	Combustible para el viaje de regreso.
combustible_total	int	Intermedia	Combustible total con reserva.
oxigeno_requerido	int	Intermedia	Oxígeno total requerido.
energia_requerida	int	Intermedia	Energía total requerida.
provisiones_requeridas	int	Intermedia	Provisiones totales requeridas.

Diagrama de flujo


INICIO
Ingresar nombre_mision
Ingresar tripulantes
Ingresar dias_viaje
combustible_ida = dias_viaje × CONSUMO_COMBUSTIBLE_DIA
combustible_regreso = dias_viaje × CONSUMO_COMBUSTIBLE_DIA
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
oxigeno_requerido = dias_viaje × CONSUMO_OXIGENO_DIA × FACTOR_IDA_REGRESO × tripulantes + RESERVA_OXIGENO
energia_requerida = dias_viaje × CONSUMO_ENERGIA_DIA × FACTOR_IDA_REGRESO + RESERVA_ENERGIA
provisiones_requeridas = dias_viaje × tripulantes × FACTOR_IDA_REGRESO + RESERVA_PROVISIONES
Mostrar resultados:
Misión: nombre_mision
Tripulantes: tripulantes
Días de viaje: dias_viaje
Combustible de ida: combustible_ida
Combustible de regreso: combustible_regreso
Combustible total: combustible_total
Oxígeno requerido: oxigeno_requerido
Energía requerida: energia_requerida
Provisiones requeridas: provisiones_requeridas
FIN

PROGRAMA DE PYTHON





mision_orbita.py | ejercicio2_combustible.py | ejercicio3

mision_orbita.py > ...

```python
    energia_requerida = (
        dias_viaje
        * CONSUMO_ENERGIA_DIA
        * FACTOR_IDA_REGRESO
        + RESERVA_ENERGIA
    )

    # Cálculo de provisiones
    provisiones_requeridas = (
        dias_viaje
        * tripulantes
        * CONSUMO_PROVISION_DIA
        * FACTOR_IDA_REGRESO
        + RESERVA_PROVISIONES
    )
    # Mostrar resultados
    print("\n--- RESULTADOS DE LA MISIÓN ---")
    print("Misión:", nombre_mision)
    print("Tripulantes:", tripulantes)
```

```python
# Mostrar resultados
print("\n--- RESULTADOS DE LA MISIÓN ---")
print("Misión:", nombre_mision)
print("Tripulantes:", tripulantes)
print("Duración estimada:", dias_viaje, "días")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Oxígeno requerido:", oxigeno_requerido, "unidades")
print("Energía requerida:", energia_requerida, "unidades")
print("Provisiones requeridas:", provisiones_requeridas, "unidades")
```

CASOS DE PRUEBA

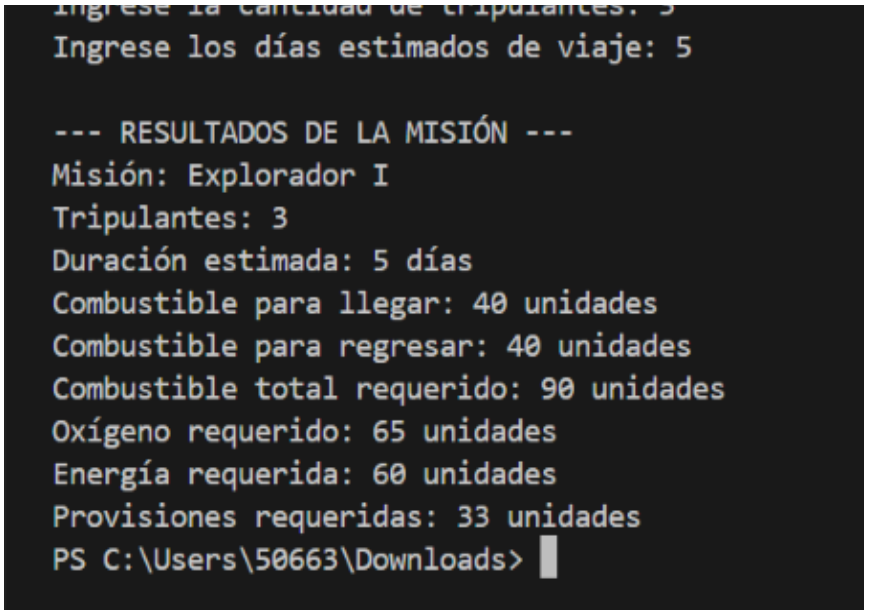
CP-01 — Caso normal

Vamos a probar:

- **Misión:** Explorador I
- **Tripulantes:** 3
- **Días:** 5

Resultado esperado:

- Combustible para llegar: **40 unidades**
- Combustible para regresar: **40 unidades**
- Combustible total requerido: **90 unidades**
- Oxígeno requerido: **65 unidades**
- Energía requerida: **60 unidades**
- Provisiones requeridas: **33 unidades**



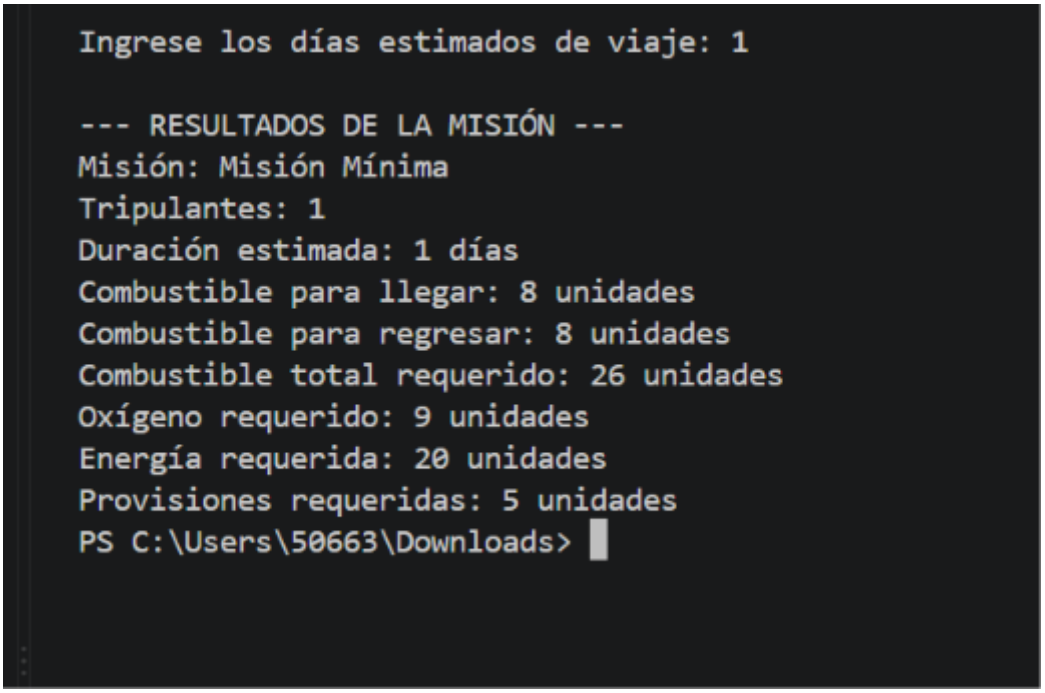
CP-02 — Caso mínimo

Vamos a probar:

- **Misión:** Misión Mínima
- **Tripulantes:** 1
- **Días:** 1

Resultado esperado:

- Combustible para llegar: **8 unidades**
- Combustible para regresar: **8 unidades**
- Combustible total requerido: **26 unidades**
- Oxígeno requerido: **9 unidades**
- Energía requerida: **20 unidades**
- Provisiones requeridas: **5 unidades**



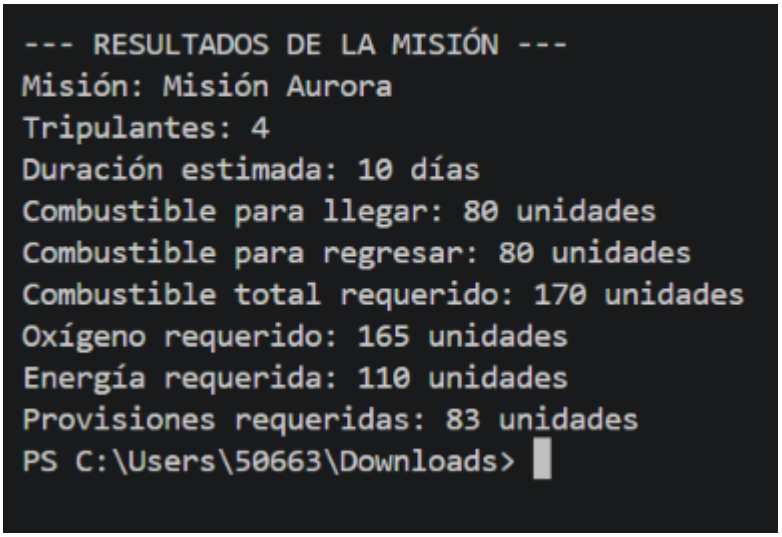
CP-03 — Caso con mayor cantidad de tripulantes y días

Vamos a probar:

- **Misión:** Misión Aurora
- **Tripulantes:** 4
- **Días:** 10

Resultado esperado:

- Combustible para llegar: **80 unidades**
- Combustible para regresar**: 80 unidades**
- Combustible total requerido: **170 unidades**
- Oxígeno requerido: **165 unidades**
- Energía requerida: **110 unidades**
- Provisiones requeridas: **83 unidades**



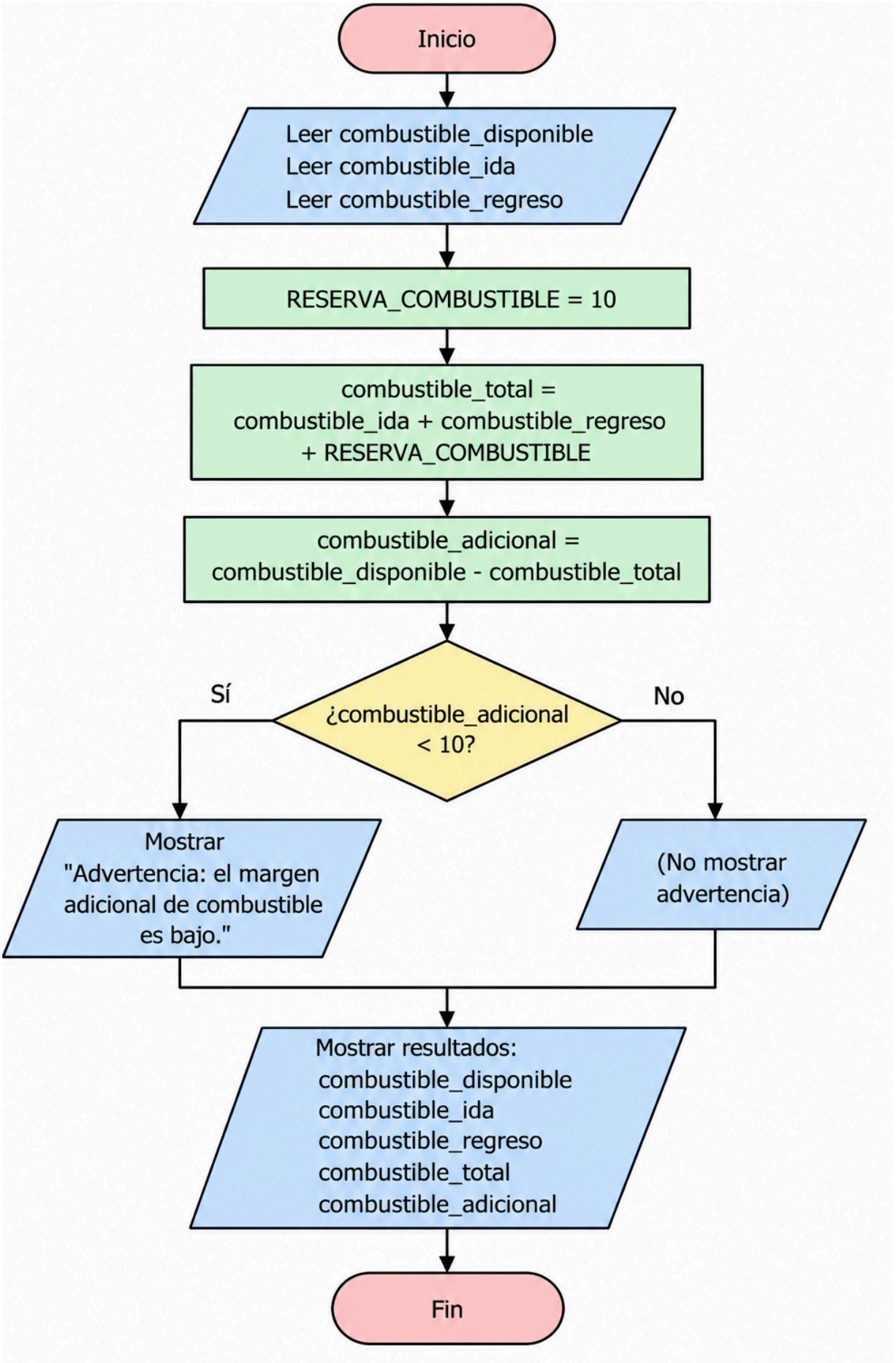
EJERCICIO 2 — Verificación de combustible

Análisis del problema

El programa permitirá verificar si una nave cuenta con suficiente combustible para completar una misión. Para ello, solicitará la cantidad de combustible disponible, el combustible requerido para el viaje de ida y el combustible requerido para el viaje de regreso. Con estos datos se calculará el combustible total requerido, incluyendo una reserva de emergencia de 10 unidades, y el combustible adicional disponible. Finalmente, mediante una estructura condicional simple, se mostrará una advertencia cuando el margen adicional de combustible sea menor que 10 unidades.

Nombre	Tipo	Clasificación	Descripción
`combustible_disponible`	int	Entrada	Cantidad de combustible disponible para la misión.
`combustible_ida`	int	Entrada	Cantidad de combustible requerida para llegar al destino.
`combustible_regreso`	int	Entrada	Cantidad de combustible requerida para regresar.
`RESERVA_COMBUSTIBLE`	int	Constante	Reserva de emergencia de 10 unidades.
`combustible_total`	int	Intermedia	Combustible total requerido para completar la misión.
`combustible_adicional`	int	Intermedia	Diferencia entre el combustible disponible y el total requerido.

Diagrama de flujo para verificar combustible


Inicio
Leer combustible_disponible
Leer combustible_ida
Leer combustible_regreso
RESERVA_COMBUSTIBLE = 10
combustible_total =
combustible_ida + combustible_regreso
+ RESERVA_COMBUSTIBLE
combustible_adicional =
combustible_disponible - combustible_total
¿combustible_adicional
< 10?
Sí
No
Mostrar
"Advertencia: el margen
adicional de combustible
es bajo."
(No mostrar
advertencia)
Mostrar resultados:
combustible_disponible
combustible_ida
combustible_regreso
combustible_total
combustible_adicional
Fin

Código en Python.

ejercicio2_combustible.py | ejercicio3_soporte_tripulacion.py | practica principios.py

ejercicio2_combustible.py > ...

```python
# Constante de la misión
RESERVA_COMBUSTIBLE = 10

# Entrada de datos
combustible_disponible = int(input("Ingrese el combustible disponible: "))
combustible_ida = int(input("Ingrese el combustible requerido para llegar: "))
combustible_regreso = int(input("Ingrese el combustible requerido para regresar: "))

# Cálculos
combustible_total = (
    combustible_ida
    + combustible_regreso
    + RESERVA_COMBUSTIBLE
)

combustible_adicional = combustible_disponible - combustible_total

# Verificación del margen de combustible
if combustible_adicional < 10:
    print("Advertencia: el margen adicional de combustible es bajo.")

# Mostrar resultados
print("\n--- RESULTADOS DE COMBUSTIBLE ---")
print("Combustible disponible:", combustible_disponible, "unidades")
print("Combustible para llegar:", combustible_ida, "unidades")
print("Combustible para regresar:", combustible_regreso, "unidades")
print("Combustible total requerido:", combustible_total, "unidades")
print("Combustible adicional:", combustible_adicional, "unidades")
```

Casos de prueba.

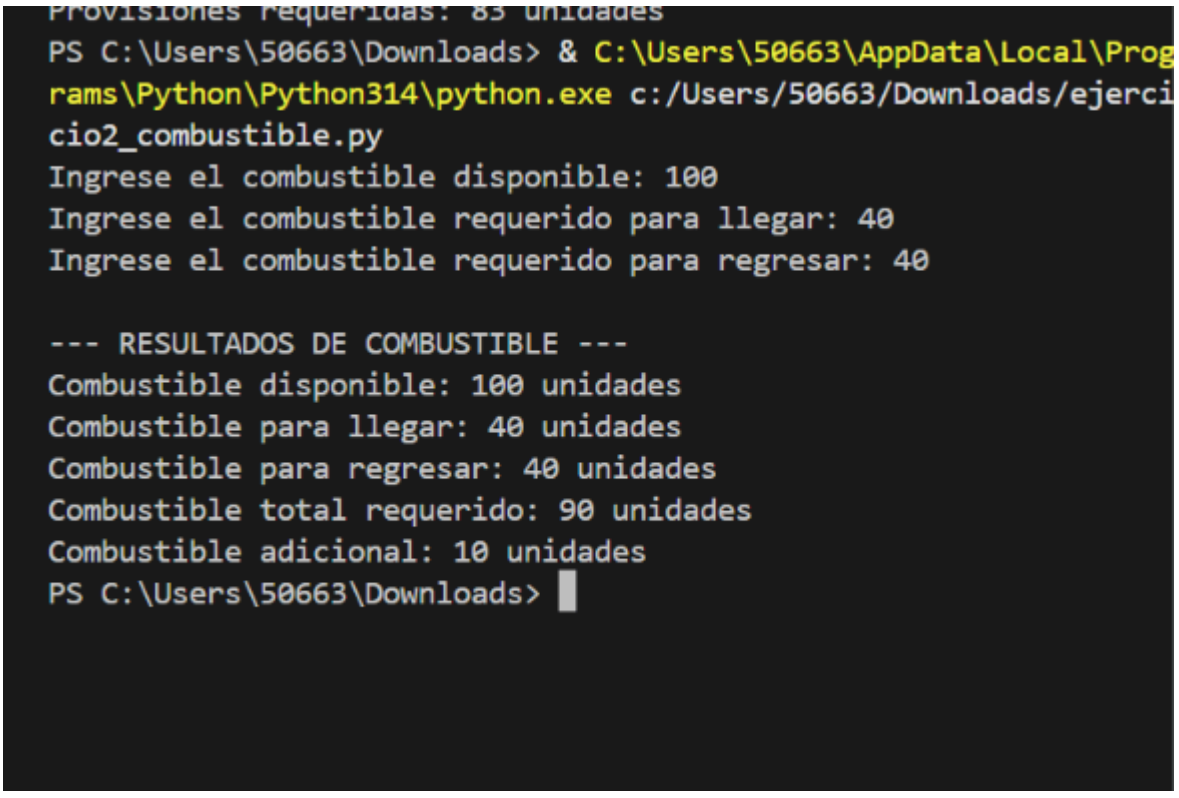
CP-01 — Caso límite

Con:

- Combustible disponible: **100**
- Combustible ida: **40**
- Combustible regreso: **40**

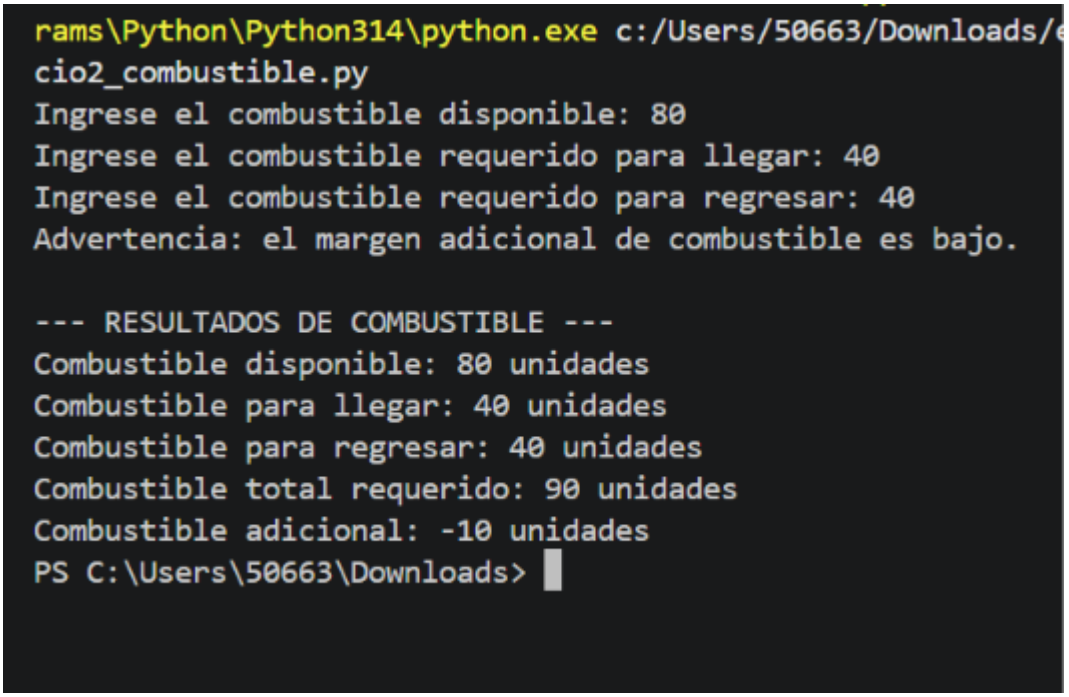
Obtuvimos:

- Total requerido: **90**
- Combustible adicional: **10**
- **No apareció la advertencia**, porque la condición es `< 10`.



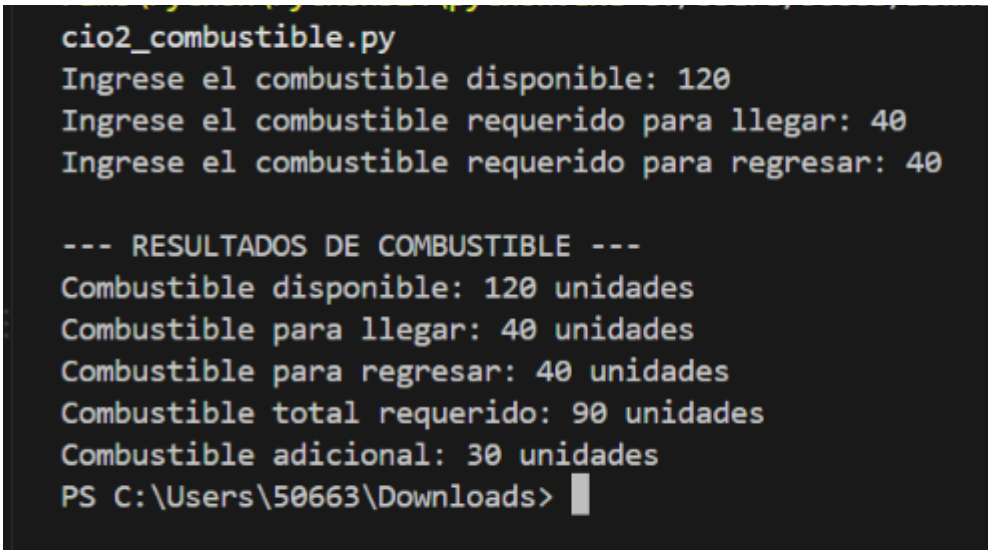
CP-02: Caso con margen insuficiente

Descripción de la prueba:
Se ingresaron 80 unidades de combustible disponible, 40 unidades requeridas para llegar y 40 unidades requeridas para regresar. El combustible total requerido es de 90 unidades y el combustible adicional resulta en **-10 unidades**. Como el margen adicional es menor que 10, el programa muestra correctamente la advertencia correspondiente.



CP-03: Caso con margen suficiente

Descripción de la prueba:
Se ingresaron 120 unidades de combustible disponible, 40 unidades requeridas para llegar y 40 unidades requeridas para regresar. El combustible total requerido es de 90 unidades y el combustible adicional resulta en **30 unidades**. Como el margen adicional es mayor que 10, el programa no muestra la advertencia.



EJERCICIO 3 — Verificación del soporte para la tripulación

Análisis del problema

El programa permitirá verificar si una nave cuenta con los recursos suficientes para la tripulación. Para realizar la verificación, solicitará la cantidad de oxígeno disponible, la cantidad de oxígeno requerida, las provisiones disponibles y las provisiones requeridas. Posteriormente, comparará ambos recursos mediante una condición lógica. Si el oxígeno disponible es suficiente y las provisiones disponibles también son suficientes, el programa indicará que la nave posee recursos suficientes para la tripulación. En caso contrario, mostrará que la nave no posee recursos suficientes.

DIAGRAMA DE FLUJO

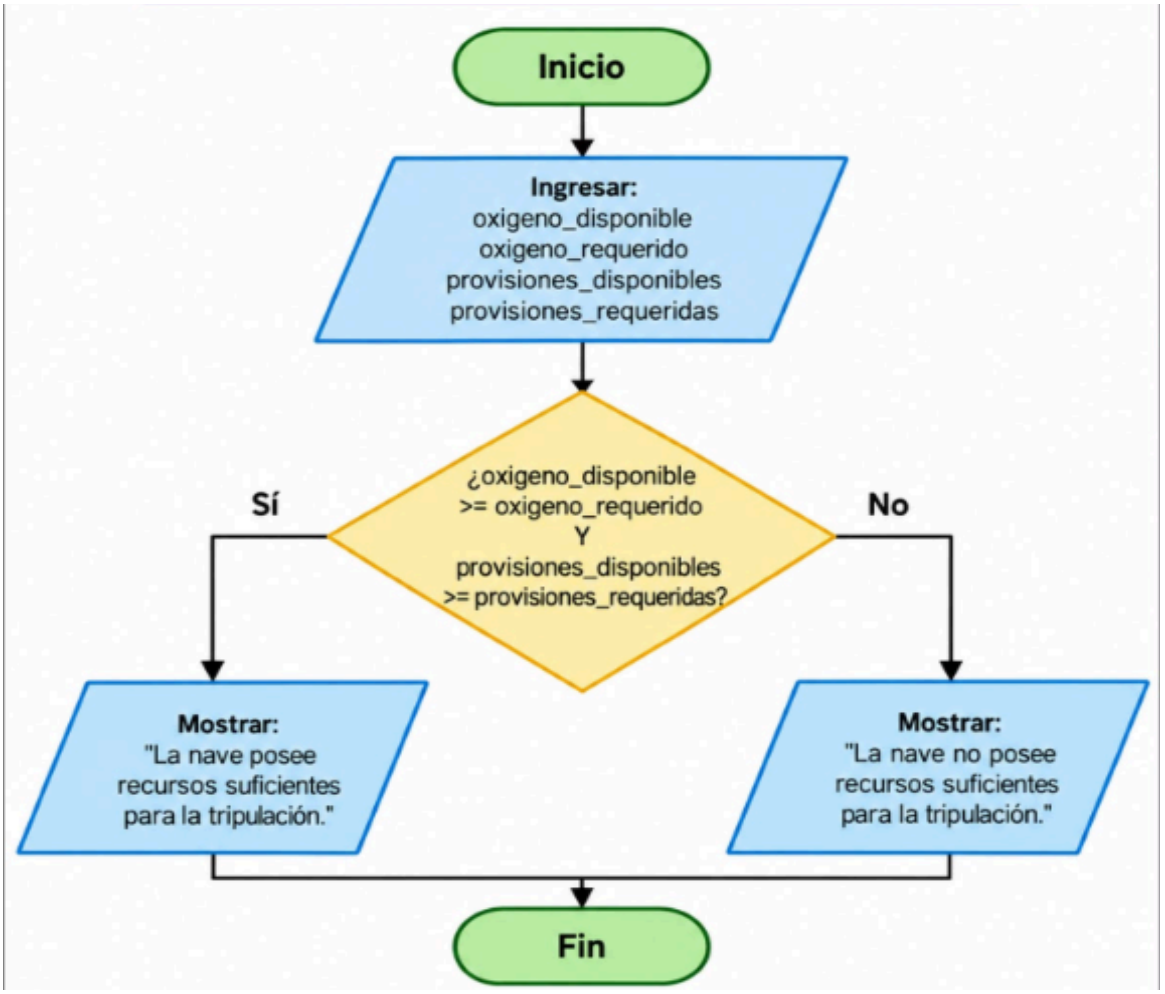
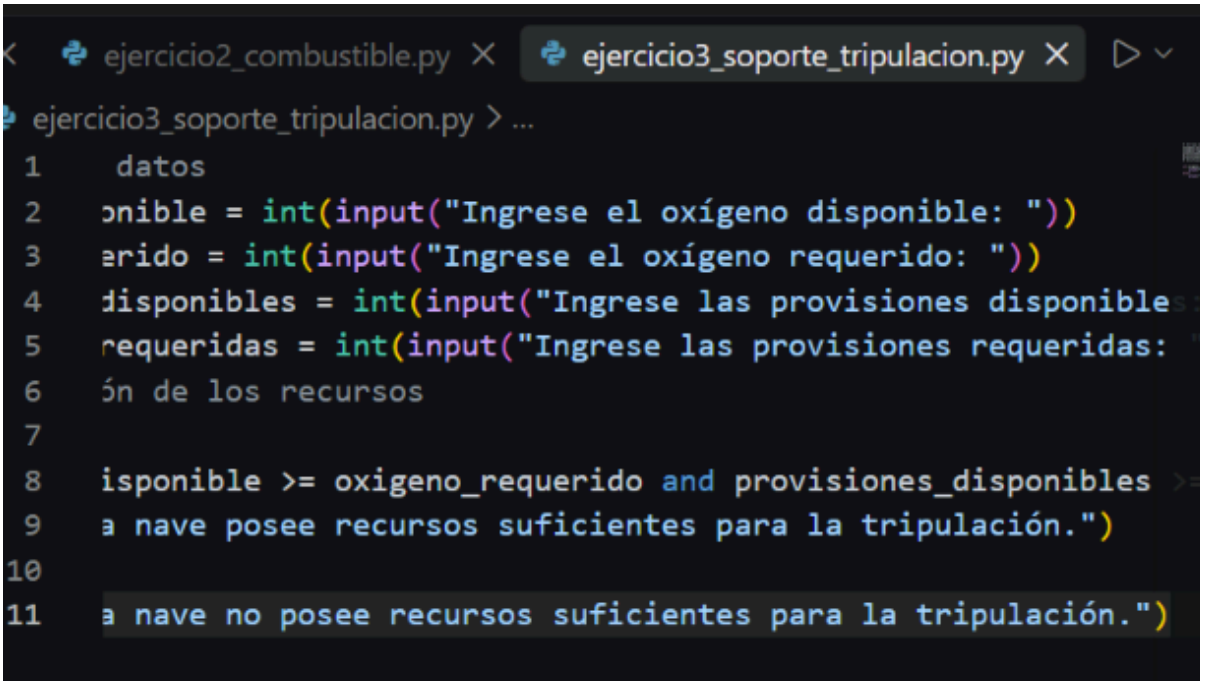


TABLA DE VARIABLES

Nombre	Tipo	Clasificación	Descripción
`oxigeno_disponible`	int	Entrada	Cantidad de oxígeno disponible para la misión.
`oxigeno_requerido`	int	Entrada	Cantidad de oxígeno necesaria para la misión.
`provisiones_disponibles`	int	Entrada	Cantidad de provisiones disponibles para la misión.
`provisiones_requeridas`	int	Entrada	Cantidad de provisiones necesarias para la misión.

Código en Python



Casos de prueba.

CP-01 — Caso con recursos suficientes

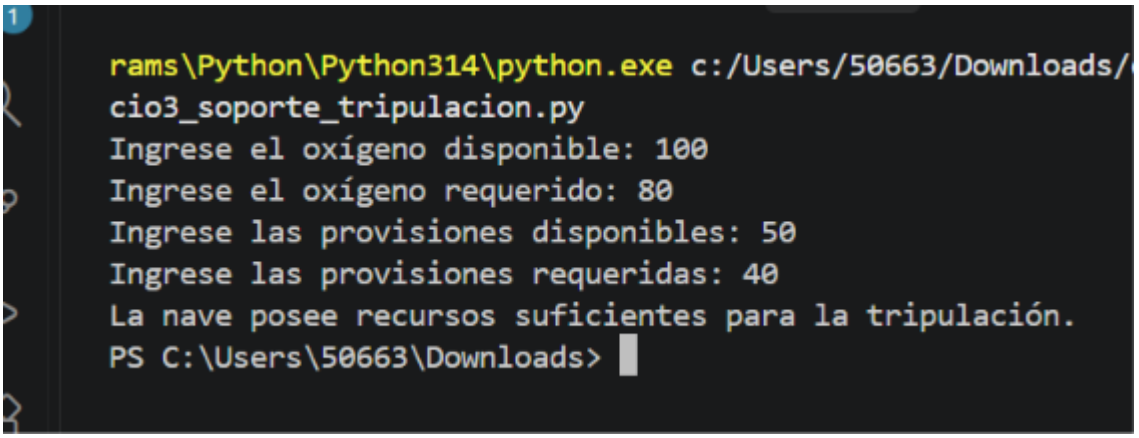
Con:

- Oxígeno disponible: **100**
- Oxígeno requerido: **80**
- Provisiones disponibles: **50**
- Provisiones requeridas: **40**

Obtuvimos:

- Las dos condiciones se cumplen.
- **La nave posee recursos suficientes para la tripulación.** ✅

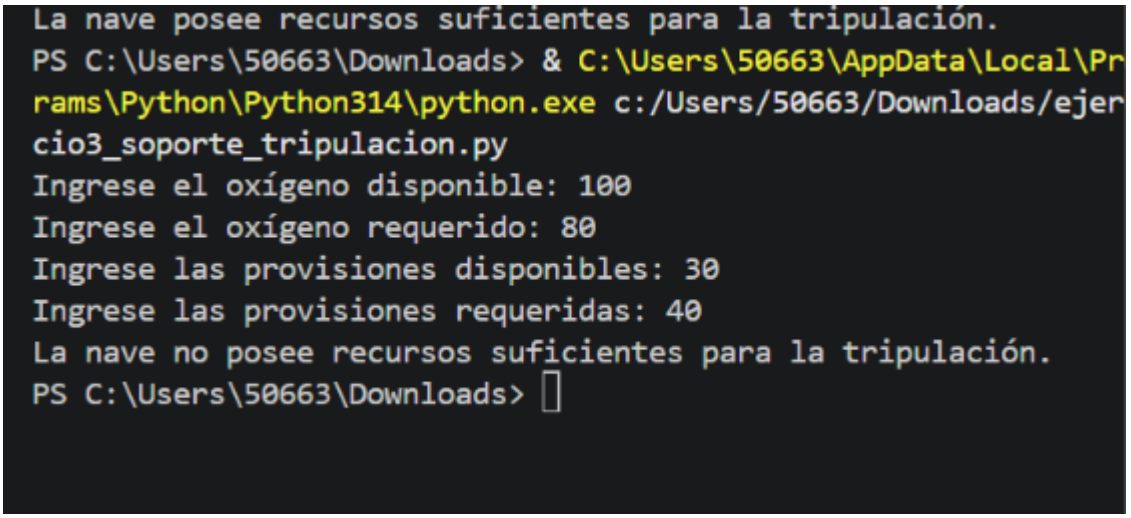
Este caso comprueba que el programa entra correctamente por la rama **Sí** cuando ambos recursos disponibles son suficientes.



CP-02 — Caso con provisiones insuficientes

Descripción de la prueba:

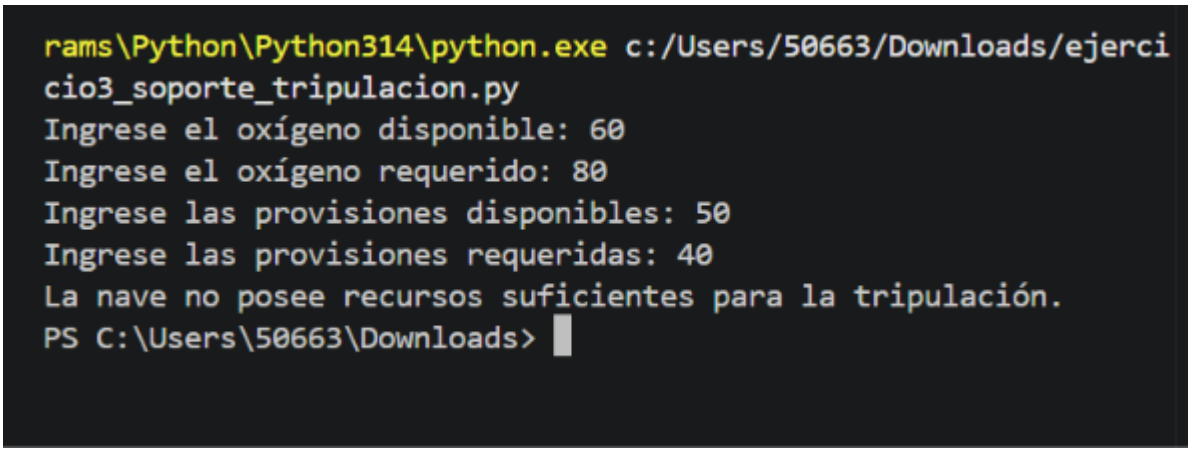
Se ingresaron 100 unidades de oxígeno disponible, 80 unidades de oxígeno requeridas, 30 unidades de provisiones disponibles y 40 unidades de provisiones requeridas. Aunque el oxígeno es suficiente, las provisiones disponibles son menores que las requeridas, por lo que el programa muestra que **la nave no posee recursos suficientes para la tripulación**.



CP-03 — Caso con oxígeno insuficiente

Descripción de la prueba:

Se ingresaron 60 unidades de oxígeno disponible, 80 unidades de oxígeno requeridas, 50 unidades de provisiones disponibles y 40 unidades de provisiones requeridas. Las provisiones son suficientes, pero el oxígeno disponible es menor que el requerido, por lo que el programa muestra que **la nave no posee recursos suficientes para la tripulación**.



EJERCICIO 4 — Autorización inicial de lanzamiento

1. Análisis del problema

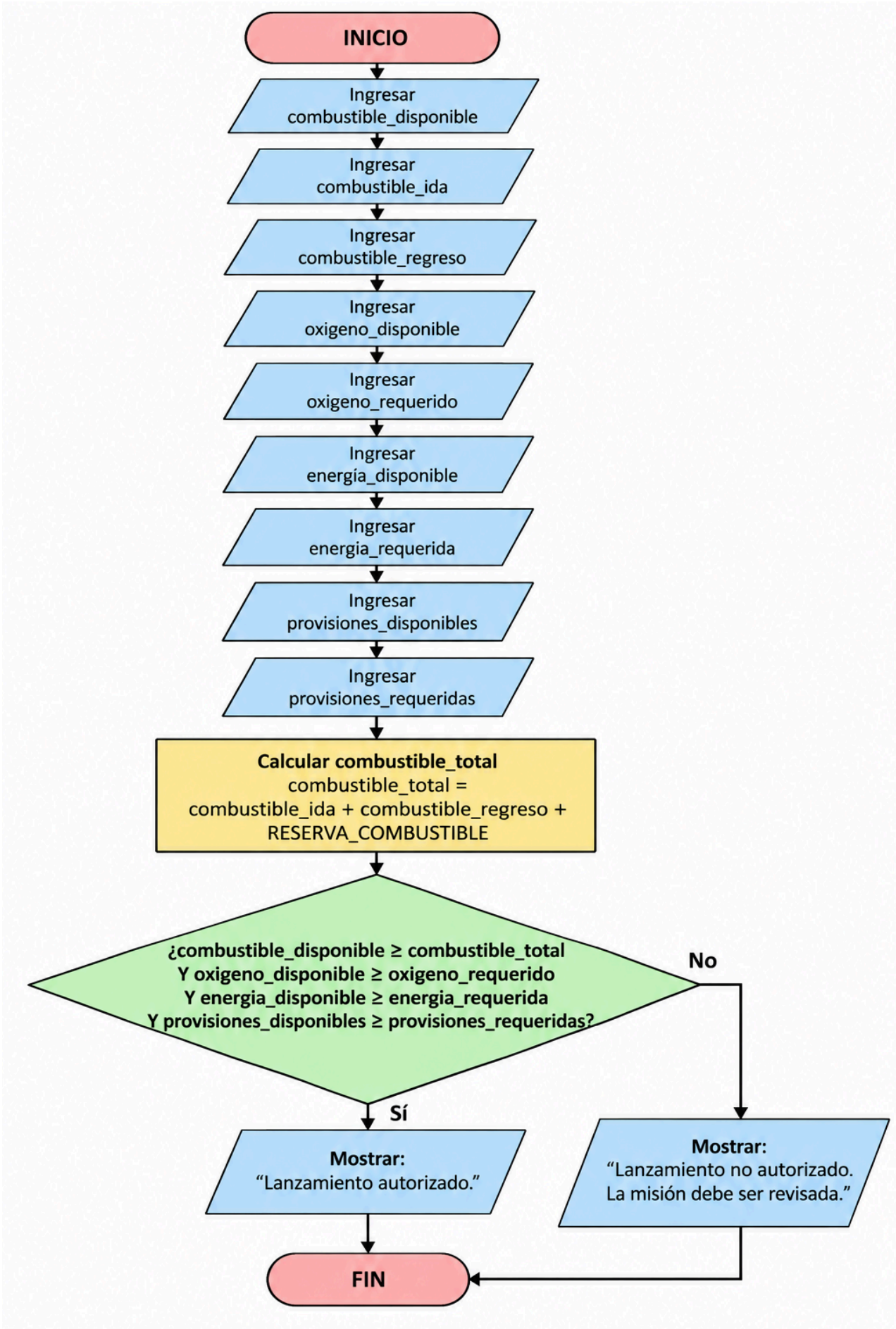
El programa permitirá verificar si una misión cumple con las condiciones necesarias para autorizar su lanzamiento. Para ello, solicitará las cantidades de combustible, oxígeno, energía y provisiones disponibles y requeridas. Primero se calculará el combustible total necesario para completar el viaje de ida y regreso, incluyendo una reserva de 10 unidades. Posteriormente, se verificará que el combustible disponible sea suficiente para cubrir el total requerido y que también sean suficientes el oxígeno, la energía y las provisiones. Si todas las condiciones se cumplen, el programa mostrará que el lanzamiento está autorizado. En caso contrario, indicará que el lanzamiento no está autorizado y que la misión debe ser revisada.

TABLA DE VARIABLES

Nombre	Tipo	Clasificación	Descripción
`combustible_disponible`	int	Entrada	Cantidad de combustible disponible para la misión.
`combustible_ida`	int	Entrada	Cantidad de combustible requerida para llegar al destino.
`combustible_regreso`	int	Entrada	Cantidad de combustible requerida para regresar.
`oxigeno_disponible`	int	Entrada	Cantidad de oxígeno disponible para la misión.
`oxigeno_requerido`	int	Entrada	Cantidad de oxígeno requerida para la misión.
`energia_disponible`	int	Entrada	Cantidad de energía disponible para la misión.

`energia_requerida`	int	Entrada	Cantidad de energía requerida para la misión.
`provisiones_disponibles`	int	Entrada	Cantidad de provisiones disponibles para la misión.
`provisiones_requeridas`	int	Entrada	Cantidad de provisiones requeridas para la misión.
`RESERVA_COMBUSTIBLE`	int	Constante	10 unidades adicionales de combustible para la misión.
`combustible_total`	int	Intermedia	Combustible total requerido para ida y regreso, incluyendo la reserva.

DIAGRAMA DE FLUJO


INICIO
Ingresar combustible_disponible
Ingresar combustible_ida
Ingresar combustible_regreso
Ingresar oxigeno_disponible
Ingresar oxigeno_requerido
Ingresar energía_disponible
Ingresar energia_requerida
Ingresar provisiones_disponibles
Ingresar provisiones_requeridas
Calcular combustible_total
combustible_total = combustible_ida + combustible_regreso + RESERVA_COMBUSTIBLE
¿combustible_disponible ≥ combustible_total Y oxigeno_disponible ≥ oxigeno_requerido Y energia_disponible ≥ energia_requerida Y provisiones_disponibles ≥ provisiones_requeridas?
No
Sí
Mostrar: "Lanzamiento autorizado."
Mostrar: "Lanzamiento no autorizado. La misión debe ser revisada."
FIN

PROGRAMA PYTHON

ejercicio4_lanzamiento.py > ...

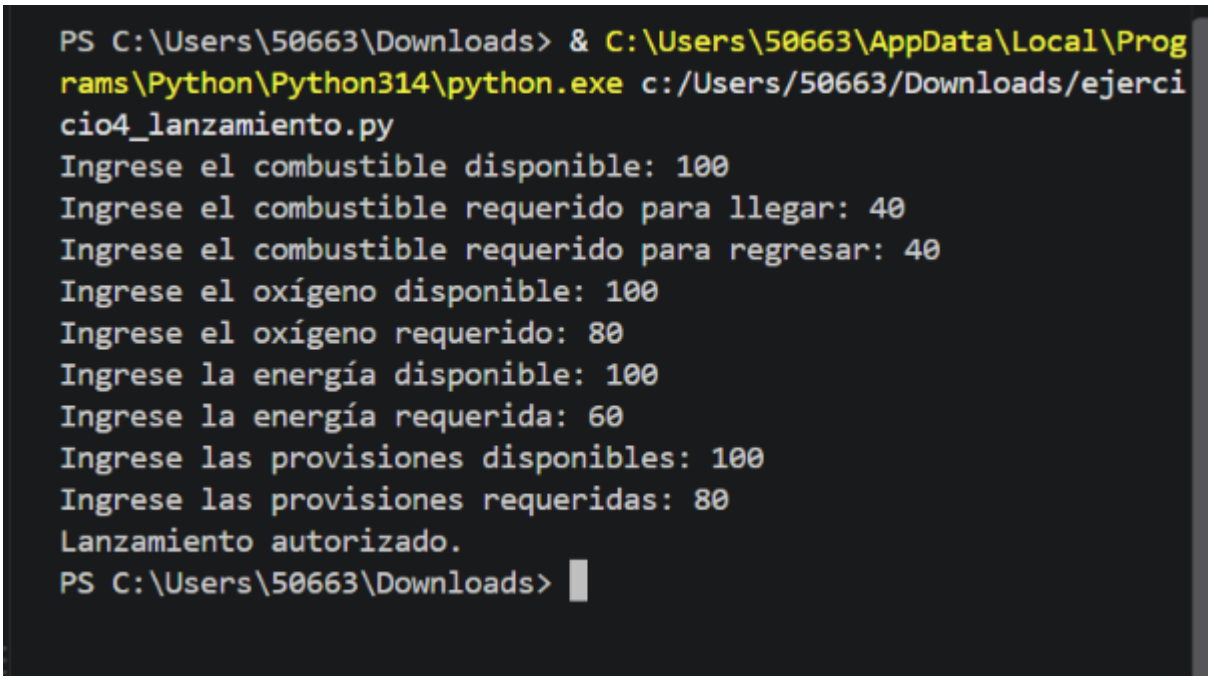
```python
# Constante de la misión
RESERVA_COMBUSTIBLE = 10
# Entrada de datos
combustible_disponible = int(input("Ingrese el combustible
combustible_ida = int(input("Ingrese el combustible requeri
combustible_regreso = int(input("Ingrese el combustible req
oxigeno_disponible = int(input("Ingrese el oxígeno disponib
oxigeno_requerido = int(input("Ingrese el oxígeno requerido
energia_disponible = int(input("Ingrese la energía disponib
energia_requerida = int(input("Ingrese la energía requerida
provisiones_disponibles = int(input("Ingrese las provisione
provisiones_requeridas = int(input("Ingrese las provisiones
# Cálculo del combustible total
combustible_total = (
    combustible_ida
    + combustible_regreso
    + RESERVA_COMBUSTIBLE
)
# Verificación de las condiciones para el lanzamiento

if (
    combustible_disponible >= combustible_total
    and oxigeno_disponible >= oxigeno_requerido
    and energia_disponible >= energia_requerida
    and provisiones_disponibles >= provisiones_requeridas
):
    print("Lanzamiento autorizado.")
else:
    print("Lanzamiento no autorizado. La misión debe ser re
```

CP-01 — Caso con lanzamiento autorizado

Descripción de la prueba:

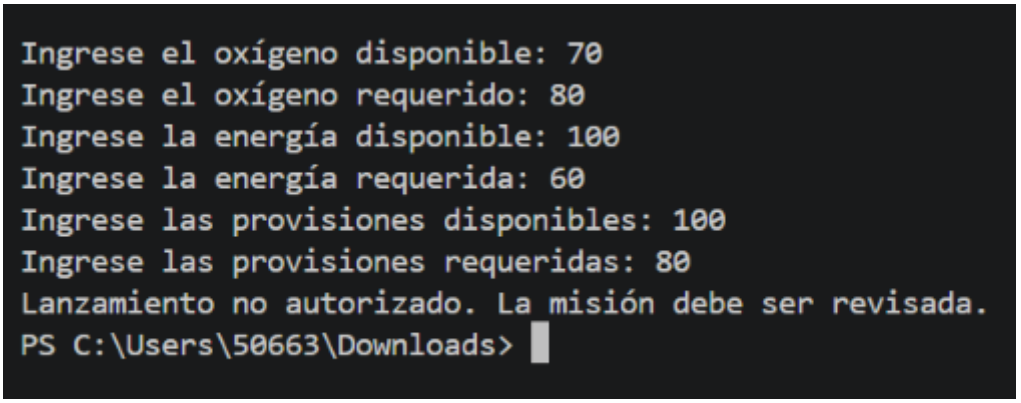
Se ingresaron 100 unidades de combustible disponible, 40 unidades requeridas para llegar, 40 unidades requeridas para regresar, 100 unidades de oxígeno disponible y 80 unidades requeridas, 100 unidades de energía disponible y 60 unidades requeridas, y 100 unidades de provisiones disponibles y 80 unidades requeridas. El combustible total requerido es de 90 unidades incluyendo la reserva. Como todas las condiciones se cumplen, el programa muestra **"Lanzamiento autorizado."**



CP-02 — Caso con oxígeno insuficiente

Descripción de la prueba:

Se ingresaron los mismos valores del caso anterior, pero con **70 unidades de oxígeno disponible y 80 unidades requeridas**. Debido a que el oxígeno disponible es menor que el requerido, no se cumplen todas las condiciones para autorizar el lanzamiento. El programa muestra **"Lanzamiento no autorizado. La misión debe ser revisada."**



CP-03 — Caso con energía insuficiente

Descripción de la prueba:

Se ingresaron 100 unidades de combustible disponible, 40 unidades requeridas para llegar, 40 unidades requeridas para regresar, 100 unidades de oxígeno disponible y 80 unidades requeridas, **50 unidades de energía disponible y 60 unidades requeridas**, y 100 unidades de provisiones disponibles y 80 unidades requeridas. Debido a que la energía disponible es menor que la requerida, el programa muestra **“Lanzamiento no autorizado. La misión debe ser revisada.”**

Resultado: ✅ Correcto.



Conclusión

El desarrollo del proyecto *Misión Órbita: Preparación de la Misión* permitió poner en práctica los conocimientos adquiridos en el curso Principios de Programación 1, mediante la creación de cuatro programas en Python orientados a la planificación y verificación de una misión espacial. A través de estos ejercicios, se trabajó con variables, constantes, tipos de datos, operaciones matemáticas y estructuras condicionales, aplicando los conceptos fundamentales de la programación para resolver situaciones relacionadas con la disponibilidad de recursos y las condiciones necesarias para autorizar un lanzamiento.

Cada uno de los programas cumplió una función específica dentro de la preparación de la misión. El primero permitió calcular las cantidades de combustible, oxígeno, energía y provisiones necesarias según el número de tripulantes y la duración del viaje. El segundo se enfocó en verificar si el combustible disponible era suficiente para cubrir el recorrido de ida y regreso, incluyendo la reserva establecida. El tercero permitió comprobar la disponibilidad de oxígeno y provisiones para la tripulación, mientras que el cuarto integró la verificación de los recursos para determinar si se cumplían las condiciones requeridas para autorizar el lanzamiento.

Asimismo, la elaboración de las tablas de variables, los diagramas de flujo y los casos de prueba contribuyó a organizar la lógica de cada solución, representar visualmente los procesos y comprobar el comportamiento de los programas ante diferentes situaciones. Las pruebas realizadas permitieron identificar las respuestas esperadas cuando los recursos eran suficientes y cuando alguna de las condiciones no se cumplía.

Finalmente, este proyecto fortaleció la comprensión de las etapas básicas del desarrollo de software, desde el análisis del problema hasta la implementación y validación del código. También permitió reconocer la importancia de escribir programas claros, mantener una estructura ordenada y verificar los resultados antes de considerar una solución terminada. Los conocimientos adquiridos constituyen una base importante para continuar desarrollando habilidades de programación y enfrentar problemas de mayor complejidad en futuros proyectos académicos.